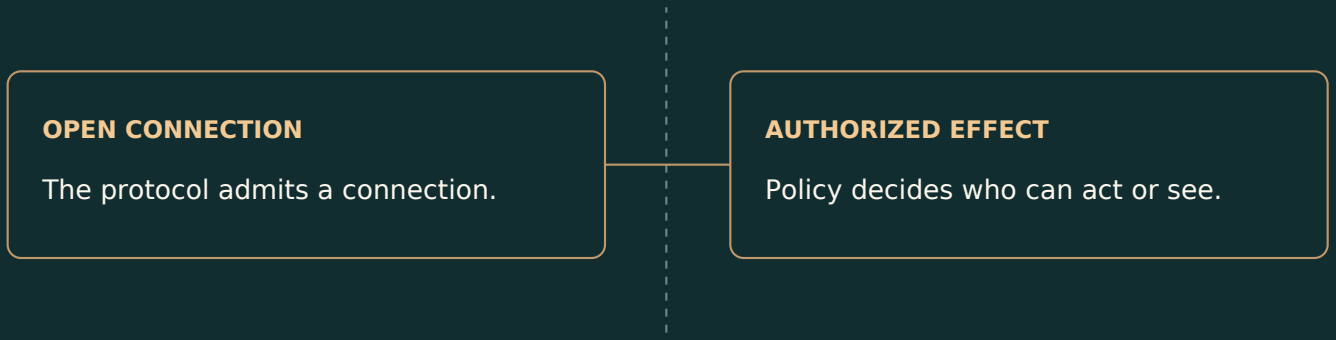
CONNECTION / MESSAGE / AUTHORITY

WebSocket Security Explained

Trace the connection, test the message, and protect real-time state



Understand the connection before testing it.
Follow the message. Prove the effect. Verify the repair.
Practice over real sockets with fictional tenant data.

Haroon Shahid

First edition / Version 1.0 / 4 November 2026

Before you begin

WebSocket Security Explained

Trace the connection, test the message, and protect real-time state

By Haroon Shahid. First edition, version 1.0. 4 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. The local lab runs real HTTP/1.1 upgrades and WebSocket messages with fictional identities and tenant data. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Signal Yard Lab.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. When the server has news first 6

02. TCP, bytes and boundaries 8

03. HTTP, TLS and reuse 10

Appendix A. Answer explanations 12

Sources and further reading 13

Introduction

Ada can edit a Pine card. Ben can watch it. Vera works for Vale and should see neither the card nor its private event. All three can open the same WebSocket endpoint. What decides which of them may act or listen?

A WebSocket gives the app a durable message path. It does not grant permanent trust. To understand or test a feature, follow the connection to its verified identity, follow the command to the commit, and follow the event to every intended recipient. When authority changes, follow it again.

Start at Chapter 1 if TCP, HTTP or event delivery feels unclear. If you already build web apps, start at Chapter 7 and use the protocol walkthrough to check your knowledge. If you test APIs, start at Chapter 19 and come back whenever a browser, credential or frame detail changes your conclusion.

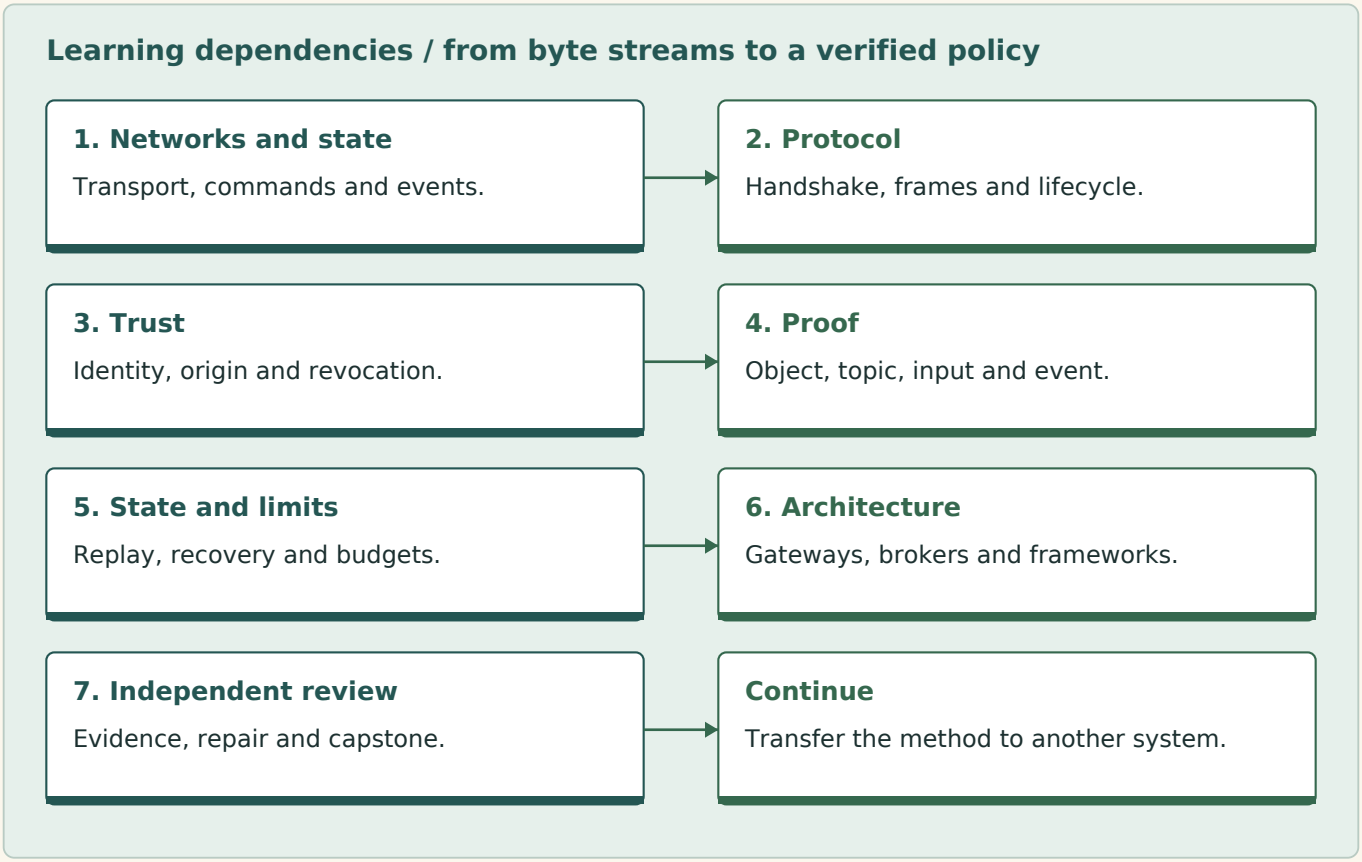
Signal Yard is a fictional work board with two tenants. Its local lab uses a real WebSocket implementation and synthetic records. Weak and repaired modes use the same fixtures. Compare them before making a claim. The strongest evidence is a meaningful state change or protected delivery for a forbidden actor, supported by a positive control.

The book progresses from the opening handshake to event delivery, reconnect, cost and distributed revocation. At the end, reproduce and repair a complete failure across several gates.

Learning roadmap

Stage	Show that you can
1. Networks and real-time state	Explain the difference between bytes, messages, commands and events.
2. Protocol	Read an opening exchange, frame and close signal.
3. Persistent trust	Bind identity and recheck action and delivery permission.
4. Discovery and proof	Prove one forbidden effect with known fixtures and a control.
5. State and limits	Predict what replay, reconnect and a slow consumer do.
6. Architecture	Trace policy and audience across gateways, brokers and workers.
7. Independent assessment	Reproduce, repair, retest and report a bounded claim.

After each part, close the book and reconstruct one decision from memory. If the explanation depends on a vague phrase such as "trusted connection," name the exact evidence and the next check.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

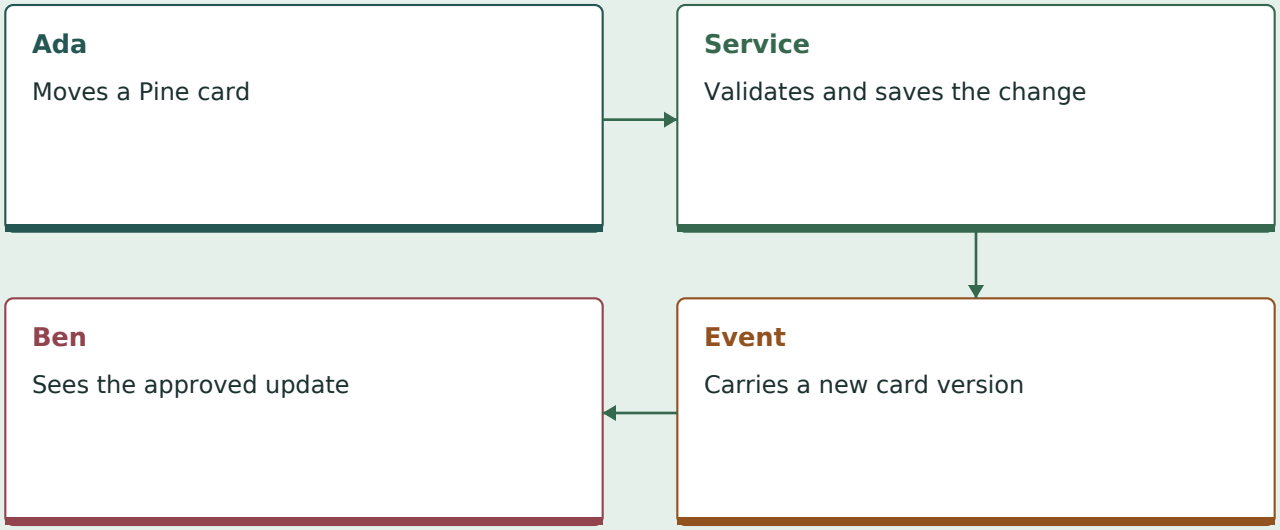
I. UNDERSTAND NETWORKS AND REAL-TIME SYSTEMS

01. When the server has news first

Learn to: Explain why a collaborative screen needs more than a one-time response.

Think of it this way: A departure board updates when the train changes. Asking at the desk every second would also work, but it wastes trips.

01A / From an action to another screen



The server can deliver an event without the second browser asking at that moment.

Start with a normal screen

Signal Yard is a fictional work board. Ada and Ben work in Pine. Vera works in Vale, a separate tenant. When Ada changes a Pine card, Ben's open screen should update. Vera should see nothing about the change. There are two decisions: who can make a change and who can learn about it.

A client is software that starts communication. A server accepts it and offers a service. One app can have several servers: a gateway that receives connections, an application that checks permissions, a database that stores cards, and a broker that carries events. A browser can be a client of the gateway while the gateway is a client of another service.

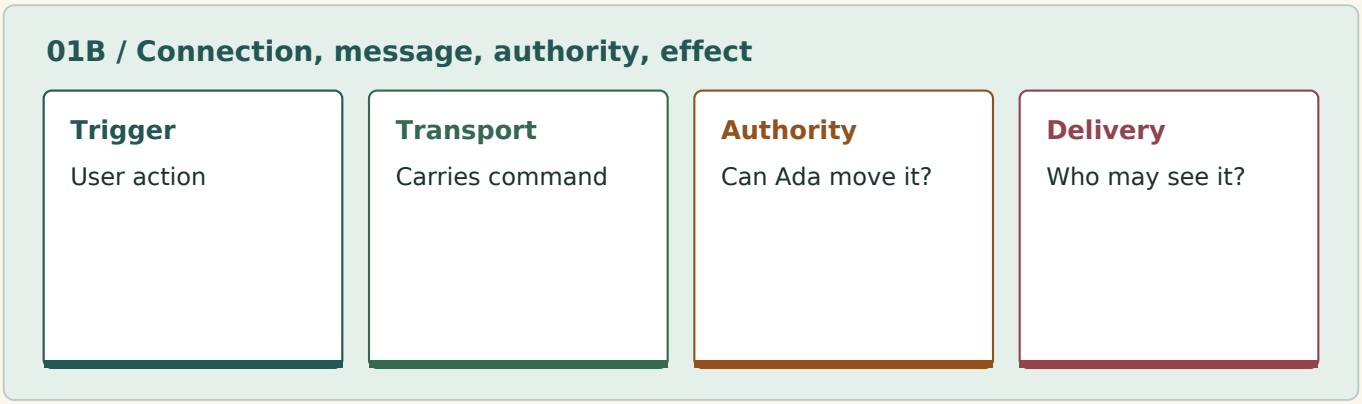
Why use a persistent channel?

The screen can repeatedly ask for changes, wait for one response, or maintain a channel that lets the server send later. A long-lived channel reduces some request overhead and can make updates arrive quickly. It also keeps state at the server: who is connected, what they subscribed to, and how much pending data they have.

A WebSocket is one way to provide that channel. It carries messages in both directions. It does not decide who owns the data, what a command means or who should receive a copy. The app must decide those things at the right time.

Practice

Draw a card update with one writer, one intended reader and one forbidden reader. Name the separate check for the write and for each delivery.



Key takeaway. A real-time action has at least two audiences: who may act and who may hear about it.

Check your understanding

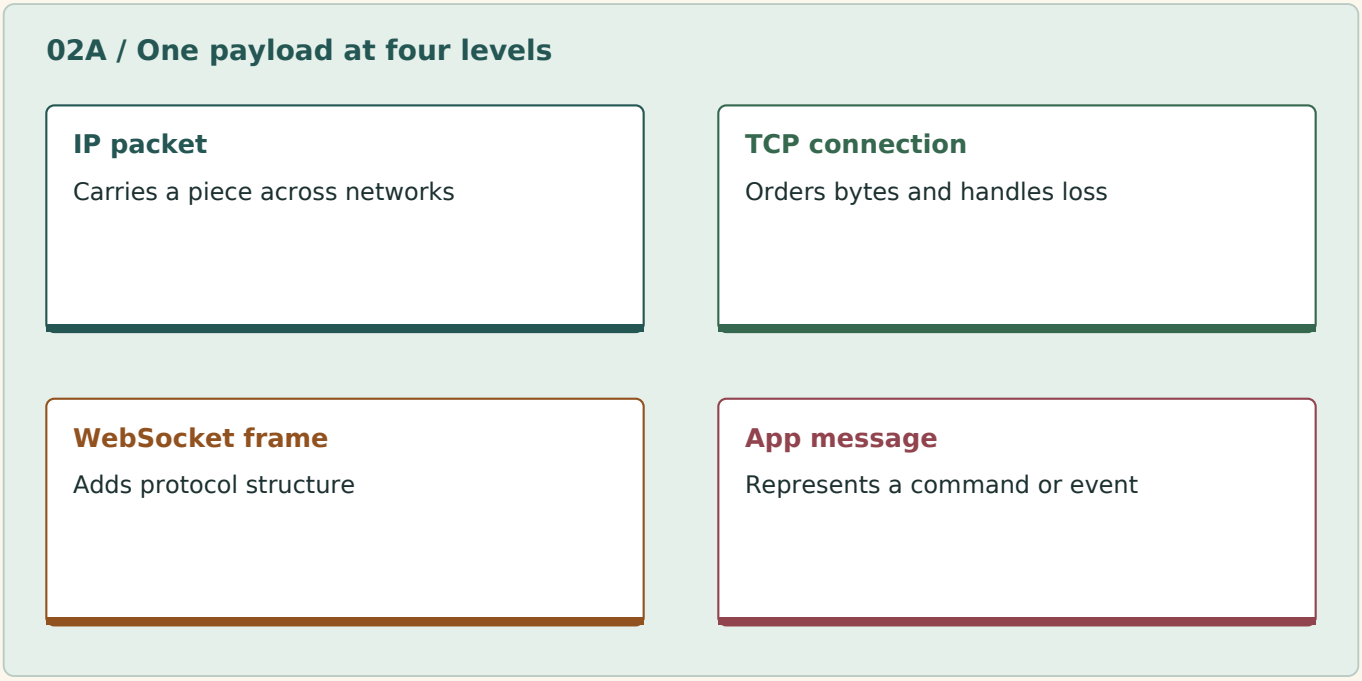
1.1 Does the server need a new HTTP request from Ben to send each WebSocket event?	**1.2** Does a connected user automatically receive every event?	**1.3** Why identify both writer and reader?

Answer explanations: Appendix A. Sources: S01, S02, S03

02. TCP, bytes and boundaries

Learn to: Distinguish IP packets, a TCP byte stream and a WebSocket message.

Think of it this way: A delivery company may split a letter into several envelopes; the reader still needs a rule for where the letter ends.



The boundaries seen by TCP and by the application are different.

Follow the data downward

A network connects machines. IP helps deliver packets to addresses. TCP adds an ordered byte stream between two endpoints. Ports help identify the communicating services. A socket is a programming interface for communication; the word is also often used for a connection in code. A TCP connection has a direction of traffic each way.

TCP presents bytes in order to an application. It does not preserve a sender's write calls as receiving message boundaries. One read might return part of a write or bytes from several writes. Network packet boundaries and TCP read boundaries are not reliable delimiters.

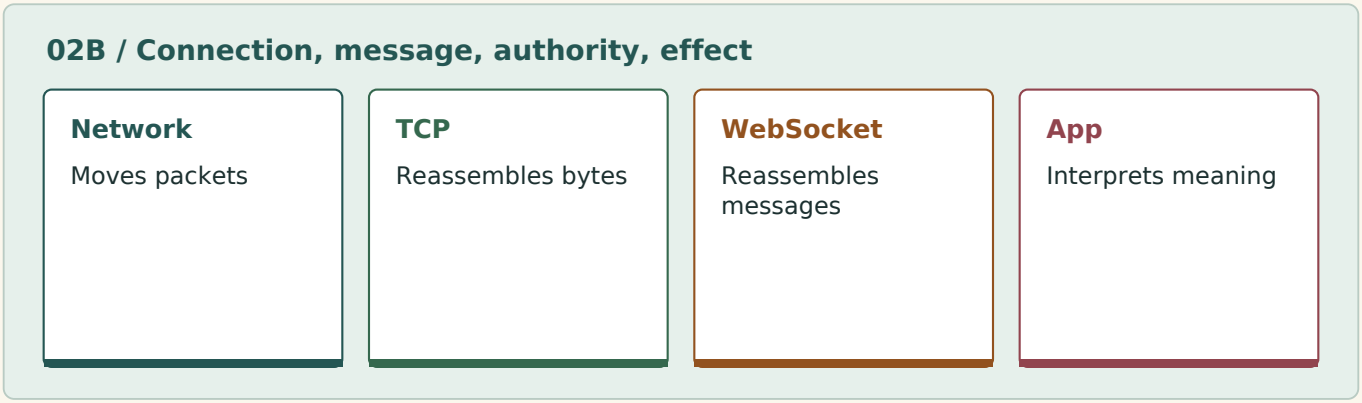
A frame is not a packet

WebSocket defines a framing layer above its underlying stream. A message can be carried in one frame or several. A frame itself may span several reads. A protocol parser reconstructs the frames, handles control traffic and delivers application messages. A browser's message event usually presents the complete message, not individual wire frames.

Ordering belongs to a particular connection and layer. A service can handle messages concurrently. Two workers may finish changes in a different order. If the connection drops, a later connection is not a continuation of TCP state. The application must define how it recognizes retries and catches up.

Practice

Draw a text message split over two WebSocket frames and three TCP reads. Mark the first point at which the handler can validate the complete command.



Key takeaway. TCP orders bytes. WebSocket defines message boundaries. The app defines what those messages mean.

Check your understanding

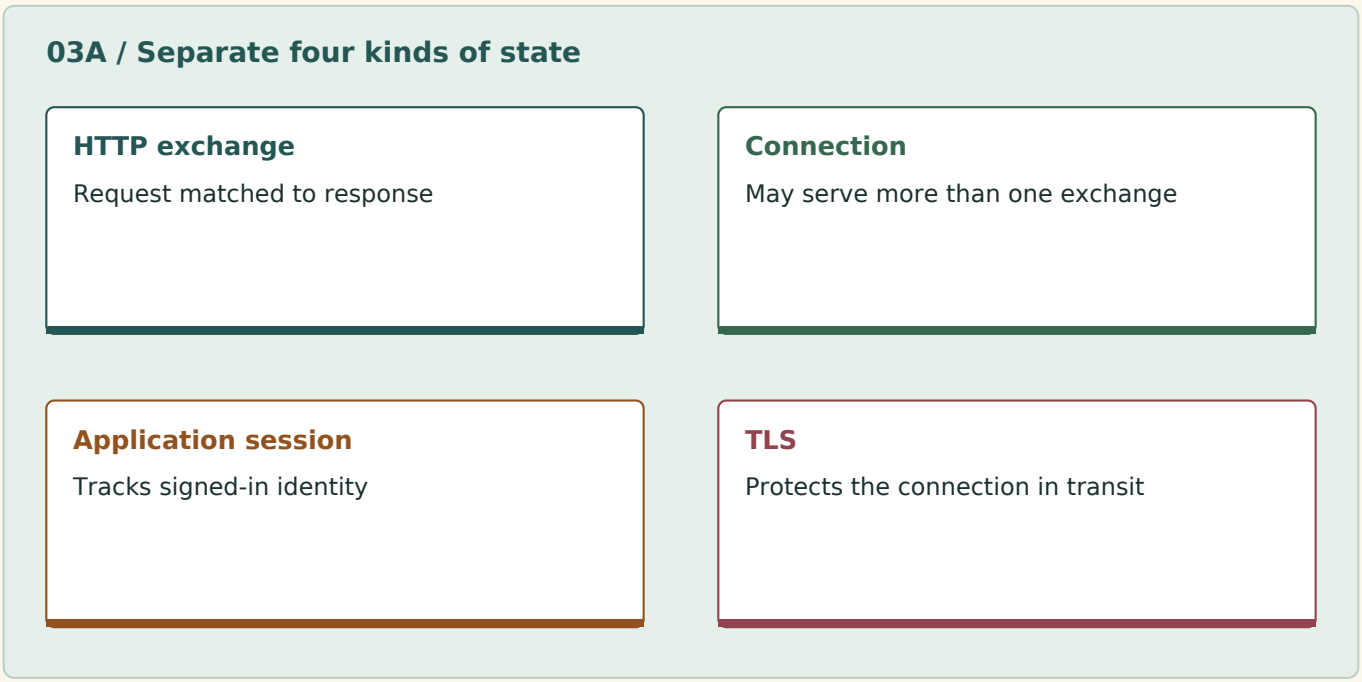
2.1 Can one TCP read contain two WebSocket frames?	**2.2** Does WebSocket guarantee global order across clients?	**2.3** When should a command be interpreted?

Answer explanations: Appendix A. Sources: S01, S04, S05

03. HTTP, TLS and reuse

Learn to: Read an HTTP exchange and distinguish connection reuse, application sessions and transport encryption.

Think of it this way: A courier can use one route for many envelopes. Each envelope can still need its own authorization.



Reusing a transport does not make later requests automatically authorized.

Requests still have their own meaning

A basic HTTP request has a method, a target, headers, and sometimes a body. The response has a status and its own headers and body. HTTP is called stateless because requests can be understood individually at the protocol level. Apps can still keep sessions, and HTTP connections can remain open across many requests. HTTP/2 and HTTP/3 can carry several streams over one connection.

A web app may use a session cookie. A cookie is a browser-managed value scoped by cookie rules. The server can look it up to identify a user. A valid session is application state; it is not the same as a TCP connection or a reused HTTP connection.

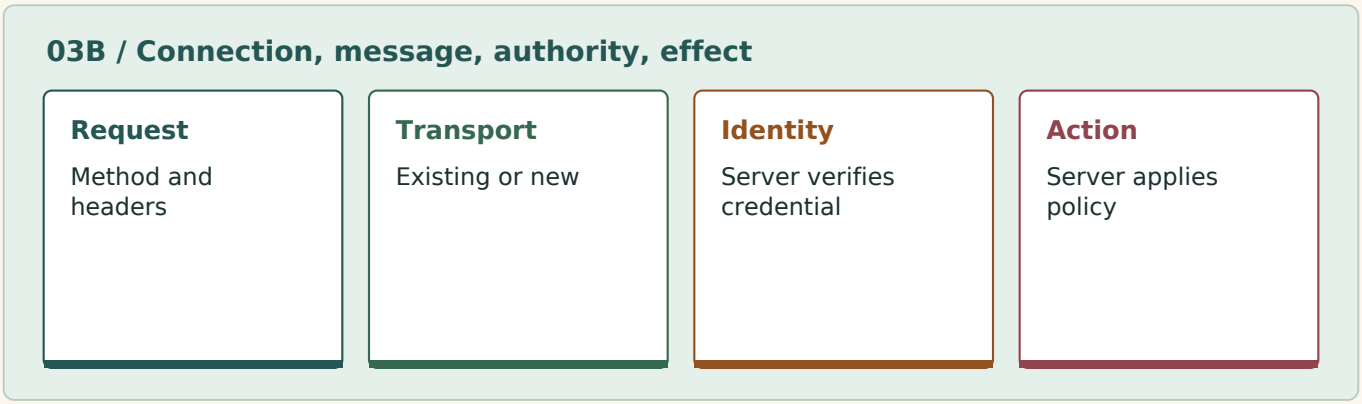
What TLS protects

HTTPS and wss use TLS when deployed correctly. TLS protects confidentiality and integrity on the encrypted hop and normally authenticates the server to the client. It does not authorize a card edit, prove that a browser tab is an approved origin, or make a false event true. If a proxy terminates TLS, inspect the next hop separately.

WebSocket commonly starts with HTTP. In HTTP/1.1 the opening handshake switches the accepted connection to the WebSocket protocol. After that, application traffic is not a series of independent HTTP requests. In HTTP/2 and HTTP/3, extended CONNECT can carry a WebSocket stream; Chapter 10 compares the mechanisms.

Practice

Label what the status, cookie, TLS certificate, and application acknowledgment each prove. Pick one fact each cannot establish.



Key takeaway. A protected transport can carry unauthorized traffic. Verify identity, action and transport separately.

Check your understanding

3.1 Does HTTP reuse make it stateful at the application level?	**3.2** What does TLS not decide?	**3.3** Does WebSocket always replace an entire HTTP connection?

Answer explanations: Appendix A. Sources: S01, S06, S07, S08, S09

Appendix A. Answer explanations

Chapter 01: When the server has news first

1.1 Does the server need a new HTTP request from Ben to send each WebSocket event? No. It can send a message on the open connection.

1.2 Does a connected user automatically receive every event? Only if the application makes that decision. Correct policy restricts the audience.

1.3 Why identify both writer and reader? Write permission and delivery permission are independent.

Chapter 02: TCP, bytes and boundaries

2.1 Can one TCP read contain two WebSocket frames? Yes. The read boundary is not a frame boundary.

2.2 Does WebSocket guarantee global order across clients? No. A connection's stream is local; business order needs a separate design.

2.3 When should a command be interpreted? After a supported parser has completed and validated the message and application input.

Chapter 03: HTTP, TLS and reuse

3.1 Does HTTP reuse make it stateful at the application level? No. Reuse and application session state are different.

3.2 What does TLS not decide? Whether a principal may send a command or receive particular data.

3.3 Does WebSocket always replace an entire HTTP connection? That describes a typical HTTP/1.1 upgrade; extended CONNECT uses a stream.

Sources and further reading

Research cutoff: 4 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

S01 - RFC 6455: The WebSocket Protocol

Opening, frame grammar, control frames, close codes and security rationale for masking.

https://www.rfc-editor.org/rfc/rfc6455.html

S02 - OWASP WebSocket Security Cheat Sheet

Check Origin, identity, permissions, message validation, resource limits and logs.

https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html

S03 - PortSwigger: WebSocket vulnerabilities

Authorized training for message manipulation and browser-driven attacks.

https://portswigger.net/web-security/websockets

S04 - RFC 9293: TCP

Transport byte stream, connection state and ordered delivery.

https://www.rfc-editor.org/rfc/rfc9293.html

S05 - WHATWG WebSockets Standard

Browser constructor, opening procedure, credentials mode, events and send buffering.

https://websockets.spec.whatwg.org/

S06 - RFC 9110: HTTP Semantics

Stateless message semantics, status, methods and hop considerations.

https://www.rfc-editor.org/rfc/rfc9110.html

S07 - RFC 8441: WebSockets with HTTP/2

Extended CONNECT and use of a single HTTP/2 stream.

https://www.rfc-editor.org/rfc/rfc8441.html

S08 - RFC 9220: WebSockets with HTTP/3

Extended CONNECT adaptation to HTTP/3.

https://www.rfc-editor.org/rfc/rfc9220.html

S09 - RFC 8446: TLS 1.3

Transport confidentiality, integrity, peer authentication and early-data caveats.

https://www.rfc-editor.org/rfc/rfc8446.html